

Lab 3: MapReduce Programming

From Theory to Hands-On Distributed Computing

Student Name: Hams Aljohani
Student ID: S20106833
Section: _____
Date: _____

Objective: Write, run, and analyze MapReduce programs using Hadoop Streaming with Python. Understand how data flows through the Map → Shuffle & Sort → Reduce pipeline.

Prerequisites: Hadoop installed and configured (Lab 1), HDFS operations (Lab 2).

Duration: 2.5 hours (Parts 1–5: guided | Part 6: deep work | Part 7: deliverables).

Tools: Terminal, Hadoop 3.x, Python 3, Text editor (nano/vim/VS Code).

Lab Structure: Part 1: Setup & YARN (10 min) → Part 2: Understanding MapReduce (15 min) → Part 3: Word Count (30 min) → Part 4: Saudi Data Analysis (30 min) → Part 5: Combiners & Optimization (20 min) → Part 6: Deep Work (30 min) → Part 7: Deliverables (10 min)

Part 1: Start Services & Verify YARN

(10 min)

MapReduce jobs run on **YARN** (Yet Another Resource Negotiator). We need both HDFS *and* YARN running.

1.1 Step 1: Start All Hadoop Services

```
>_ Start HDFS and YARN

# Start HDFS (from Lab 2)
start-dfs.sh

# Start YARN (resource manager for MapReduce)
start-yarn.sh

# Verify all processes
jps
```

You should see **at least 5 processes**: NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager.

1.2 Step 2: Access Web UIs

- HDFS NameNode: <http://localhost:9870>
- YARN ResourceManager: <http://localhost:8088>

Task 1.1

- Run `jps` and list all running processes: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager
- Take a screenshot of the **YARN ResourceManager Web UI** (<http://localhost:8088>).

Common Issue

If YARN fails to start, check the logs:

```
cat $HADOOP_HOME/logs/hadoop-*-resourcemanager-*.log | tail -30
```

Make sure `yarn-site.xml` has `mapreduce_shuffle` configured (see Lab 1).

1.3 Step 3: Find Hadoop Streaming JAR

Hadoop Streaming lets us write MapReduce in **any language** (Python, Ruby, etc.) instead of Java.

>_ Locate Streaming JAR

```
# Find the Hadoop Streaming JAR file
find $HADOOP_HOME -name "hadoop-streaming*.jar" -type f

# Save it as a variable for later use
export STREAMING_JAR=$(find $HADOOP_HOME -name "hadoop-streaming*.jar"
    " -type f | head -1)
echo $STREAMING_JAR
```

Task 1.2

Record the full path to your Hadoop Streaming JAR:

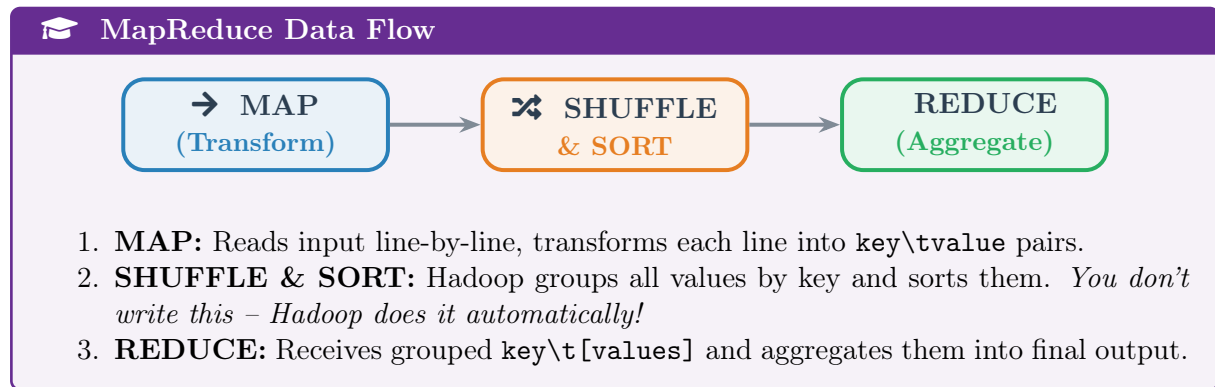
/opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar

Part 2: Understanding the MapReduce Pipeline

(15 min)

Before writing code, let's understand *exactly* how data flows through MapReduce.

2.1 The Three Phases



2.2 Word Count Example – Pencil Trace

Given this input file with 2 lines:

```
hello world hello
big data big
```

MAP Output	SHUFFLE & SORT	REDUCE Output
hello\t1	big\t[1, 1]	big\t2
world\t1	data\t[1]	data\t1
hello\t1	hello\t[1, 1]	hello\t2
big\t1	world\t[1]	world\t1
data\t1		
big\t1		

Task 2.1 – Pencil Trace

Trace the MapReduce execution for this input:

```
spark hadoop spark
hadoop hdfs hadoop spark
```

Fill in the table:

MAP Output	SHUFFLE & SORT	REDUCE Output
spark 1 hadoop 1 spark 1 hadoop 1 hdfs 1 hadoop 1 spark 1	hadoop [1,1,1] hdfs [1] spark [1,1,1]	hadoop 3 hdfs 1 spark 3

Part 3: Your First MapReduce Program: Word Count (30 min)

3.1 Step 1: Create the Input Data

>_ Prepare Input File

```
# Create input text file
cat > ~/wordcount_input.txt << 'EOF'
Big Data Analytics is transforming how we understand the world
Hadoop provides a framework for distributed storage and processing
MapReduce divides work across multiple machines in a cluster
Saudi Vision 2030 leverages Big Data for smart cities
Effat University teaches Big Data Analytics and Hadoop
Data is the new oil and Big Data is the refinery
EOF

# Upload to HDFS
hdfs dfs -mkdir -p /user/student/lab3/input
hdfs dfs -put ~/wordcount_input.txt /user/student/lab3/input/

# Verify
hdfs dfs -cat /user/student/lab3/input/wordcount_input.txt
```

3.2 Step 2: Write the Mapper (Python)

The mapper reads **stdin** line by line, splits each line into words, and outputs **word\t1**.

>_ Create mapper.py

```
cat > ~/mapper.py << 'PYEOF'
#!/usr/bin/env python3
"""
MapReduce Mapper: Word Count
Input:  lines of text (from stdin)
Output: word\t1 for each word (to stdout)
"""
import sys

for line in sys.stdin:
    # Remove leading/trailing whitespace
    line = line.strip()
    # Split line into words
    words = line.split()
    # Emit each word with count 1
    for word in words:
        # Convert to lowercase for consistency
        print(f"{word.lower()}\t1")
PYEOF

# Make it executable
chmod +x ~/mapper.py
```

✓ Key Insight

The mapper communicates via **stdin/stdout** – it reads lines from standard input and writes key-value pairs to standard output, separated by a tab (**\t**). This is the Hadoop Streaming protocol.

3.3 Step 3: Write the Reducer (Python)

The reducer receives **sorted** key-value pairs and sums up the counts for each word.

```
>_ Create reducer.py

cat > ~/reducer.py << 'PYEOF'
#!/usr/bin/env python3
"""
MapReduce Reducer: Word Count
Input:  sorted word\tcount pairs (from stdin)
Output: word\ttotal_count (to stdout)
"""
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    line = line.strip()
    # Parse the key-value pair
    try:
        word, count = line.split('\t', 1)
        count = int(count)
    except ValueError:
        continue # Skip malformed lines

    # If same word, accumulate count
    if word == current_word:
        current_count += count
    else:
        # Output the previous word's total
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
            current_word = word
            current_count = count

# Don't forget the last word!
if current_word is not None:
    print(f"{current_word}\t{current_count}")
PYEOF

# Make it executable
chmod +x ~/reducer.py
```

3.4 Step 4: Test Locally Before Hadoop

Always test locally first! This saves time and catches bugs before submitting to the cluster.

>_ Local Testing Pipeline

```
# Test mapper alone
echo "hello world hello" | python3 ~/mapper.py

# Expected output:
# hello 1
# world 1
# hello 1

# Test full pipeline: mapper | sort | reducer
cat ~/wordcount_input.txt | python3 ~/mapper.py | sort | python3 ~/
reducer.py

# The sort command simulates Hadoop's Shuffle & Sort phase!
```

Task 3.1 – Local Testing

- a) Run the local pipeline above. List the **top 5 words** by count: data → 5, big → 4, and → 3, is → 3, the → 3
The top 5 most frequent words are: data (5), big (4), and (3), is (3), the (3).
- b) What is the count for the word “big”? The word “big” appears 4 times.
- c) Why do we need the **sort** command between mapper and reducer?
 The sort command simulates Hadoop's Shuffle & Sort phase. It groups identical keys together so the reducer can correctly aggregate the values.

3.5 Step 5: Run on Hadoop

>_ Submit MapReduce Job to Hadoop

```
# Remove output directory if it exists (Hadoop won't overwrite)
hdfs dfs -rm -r /user/student/lab3/output 2>/dev/null

# Run the MapReduce job using Hadoop Streaming
hadoop jar $STREAMING_JAR \
  -input /user/student/lab3/input/wordcount_input.txt \
  -output /user/student/lab3/output \
  -mapper "python3 mapper.py" \
  -reducer "python3 reducer.py" \
  -file ~/mapper.py \
  -file ~/reducer.py

# Check the job output
hdfs dfs -ls /user/student/lab3/output/

# View results
hdfs dfs -cat /user/student/lab3/output/part-00000
```

⚠ Output Directory Must Not Exist

Hadoop will **refuse to run** if the output directory already exists. Always delete it first with `hdfs dfs -rm -r <output-dir>` before re-running a job.

Task 3.2 – Hadoop Execution

a) Take a screenshot of the **YARN Web UI** (<http://localhost:8088>) showing your completed job.

b) List all files in the output directory. What files did Hadoop create?

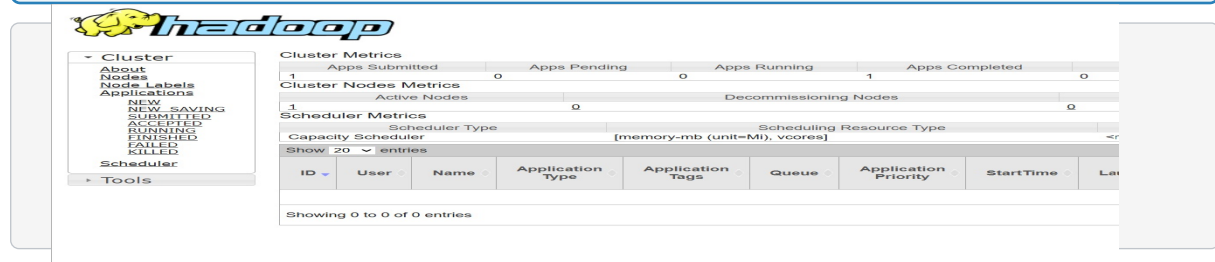
- `_SUCCESS`
- `part-00000`

c) Show the first 10 lines of the output:

```
2030 1
a 2
across 1
analytics 2
and 3
big 4
cities 1
cluster 1
data 5
distributed 1
```

d) What is the purpose of the `_SUCCESS` file?

It indicates the MapReduce job completed successfully without errors.



3.6 Step 6: Sort Results by Count

> Find Top Words

```
# Sort by count (descending) to find most frequent words
hdfs dfs -cat /user/student/lab3/output/part-00000 | \
  sort -t$'\t' -k2 -nr | head -10

# Count total unique words
hdfs dfs -cat /user/student/lab3/output/part-00000 | wc -l
```

Part 4: Real-World Exercise: Saudi Population Analysis (30 min)

In Lab 2, we uploaded a Saudi cities dataset. Now we'll analyze it using MapReduce!

4.1 Step 1: Prepare the Dataset

>_ Create Saudi Dataset

```
# Create an extended Saudi cities dataset
cat > ~/saudi_cities.csv << 'EOF'
city,region,population,area_km2,hospitals,universities
Riyadh,Central,7500000,1798,85,25
Jeddah,Western,4700000,1686,62,18
Makkah,Western,2200000,760,45,8
Madinah,Western,1500000,589,38,6
Dammam,Eastern,1300000,800,35,10
Khobar,Eastern,650000,750,20,5
Tabuk,Northern,600000,780,15,3
Abha,Southern,500000,370,18,4
Taif,Western,700000,321,22,5
Buraidah,Central,600000,980,12,3
Hail,Northern,450000,620,10,2
Najran,Southern,400000,550,8,2
Jazan,Southern,350000,480,12,3
Yanbu,Western,300000,310,10,2
Jubail,Eastern,500000,520,15,4
EOF

# Upload to HDFS
hdfs dfs -mkdir -p /user/student/lab3/saudi_input
hdfs dfs -put ~/saudi_cities.csv /user/student/lab3/saudi_input/
```

4.2 Step 2: MapReduce – Total Population Per Region

>_ Create region_mapper.py

```
cat > ~/region_mapper.py << 'PYEOF'
#!/usr/bin/env python3
"""
Mapper: Extract region and population from CSV
Input:  CSV lines (city,region,population,area,hospitals,universities
)
Output: region\tpopulation
"""
import sys

for line in sys.stdin:
    line = line.strip()
    # Skip the header line
    if line.startswith("city,"):
        continue
    parts = line.split(",")
    if len(parts) >= 3:
        region = parts[1]
```

```

        try:
            population = int(parts[2])
            print(f"{region}\t{population}")
        except ValueError:
            continue
PYEOF

chmod +x ~/region_mapper.py

```

```

>_ Create region_reducer.py

cat > ~/region_reducer.py << 'PYEOF'
#!/usr/bin/env python3
"""
Reducer: Sum population per region
Input:  sorted region\tpopulation pairs
Output: region\ttotal_population
"""
import sys

current_region = None
total_population = 0

for line in sys.stdin:
    line = line.strip()
    try:
        region, population = line.split('\t', 1)
        population = int(population)
    except ValueError:
        continue

    if region == current_region:
        total_population += population
    else:
        if current_region is not None:
            print(f"{current_region}\t{total_population}")
            current_region = region
            total_population = population

if current_region is not None:
    print(f"{current_region}\t{total_population}")
PYEOF

chmod +x ~/region_reducer.py

```

4.3 Step 3: Test Locally, Then Run on Hadoop

```

>_ Test and Run

# Test locally
cat ~/saudi_cities.csv | python3 ~/region_mapper.py | sort | \
python3 ~/region_reducer.py

# Run on Hadoop
hdfs dfs -rm -r /user/student/lab3/region_output 2>/dev/null

```

```
hadoop jar $STREAMING_JAR \
  -input /user/student/lab3/saudi_input/saudi_cities.csv \
  -output /user/student/lab3/region_output \
  -mapper "python3 region_mapper.py" \
  -reducer "python3 region_reducer.py" \
  -file ~/region_mapper.py \
  -file ~/region_reducer.py

# View results
hdfs dfs -cat /user/student/lab3/region_output/part-00000
```

Task 4.1 – Saudi Population Analysis

a) What is the total population of each region?

Region	Total Population
Central	$7500000 + 600000 = 8100000$
Eastern	$1300000 + 650000 + 500000 = 2450000$
Northern	$600000 + 450000 = 1050000$
Southern	$500000 + 400000 + 350000 = 1250000$
Western	$4700000 + 2200000 + 1500000 + 700000 + 300000 = 9400000$

b) Which region has the largest population? Western (9,400,000)

c) Show the command and output of your local test:

```
hams33443344@DESKTOP-IA041L8:~$ cat ~/saudi_cities.csv | python3 ~/region_mapper.py | sort | python3 ~/region_reducer.py
Central 8100000
Eastern 2450000
Northern 1050000
Southern 1250000
Western 9400000
hams33443344@DESKTOP-IA041L8:~$ |
```

4.4 Step 4: MapReduce – Average Hospital Density

Now write a MapReduce program to calculate: **average number of hospitals per city in each region.**

Task 4.2 – Write Your Own MapReduce

Write a mapper (`hospital_mapper.py`) and reducer (`hospital_reducer.py`) to compute the average hospitals per city for each region.

Hints:

- Mapper output: `region\thospitals`
- Reducer: Track both sum and count, then compute average

a) Write your mapper code:

```
cat > ~/hospital_mapper.py << 'PYEOF'
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    line = line.strip()
    if line.startswith("city,") or not line:
        continue
    parts = line.split(",")
    if len(parts) >= 5:
        region = parts[1]
        hospitals = parts[4]
        print(f"{region}\t{hospitals}")
PYEOF
chmod +x ~/hospital_mapper.py
```

b) Write your reducer code:

```

cat > ~/hospital_reducer.py << 'PYEOF'
#!/usr/bin/env python3
import sys
current_region = None
total_hospitals = 0
city_count = 0

for line in sys.stdin:
    line = line.strip()
    try:
        region, hospitals = line.split('\t', 1)
        hospitals = int(hospitals)
    except ValueError:
        continue

    if region == current_region:
        total_hospitals += hospitals
        city_count += 1
    else:
        if current_region is not None:
            avg = total_hospitals / city_count
            print("{}{}{}\t{:.2f}"
                  .format(current_region, "\t", avg, 2f))
            current_region = region
            total_hospitals = hospitals
            city_count = 1

if current_region is not None:
    avg = total_hospitals / city_count
    print("{}{}{}\t{:.2f}"
          .format(current_region, "\t", avg, 2f))
PYEOF
chmod +x ~/hospital_reducer.py

```

c) Show the output (region → average hospitals):

```

cat ~/saudi_cities.csv | python3 ~/hospital_mapper.py | sort | python3 ~/hospital_reducer.py

```

Output:

- **Central:** 48.50
- **Eastern:** 23.33
- **Northern:** 12.50
- **Southern:** 12.67
- **Western:** 35.40

```

hams3344344@DESKTOP-IA041L8:~$ cat ~/saudi_cities.csv | python3 ~/hospital_mapper.py | sort | python3 ~/hospital_reducer.py
Central 48.50
Eastern 23.33
Northern 12.50
Southern 12.67
Western 35.40
hams3344344@DESKTOP-IA041L8:~$

```

Part 5: Combiners & Optimization

(20 min)

5.1 What Is a Combiner?

Combiner: Local Pre-Aggregation

A **combiner** is a “mini-reducer” that runs **on the mapper’s machine** before data is sent across the network. It reduces network traffic by pre-aggregating locally.



5.2 Example: With vs. Without Combiner

Suppose a mapper on Node A produces:

```
big    1
big    1
big    1
data   1
data   1
```

	Without Combiner	With Combiner
Records sent to reducer	5 records	2 records
Network data	big,1 big,1 big,1 data,1 data,1	big,3 data,2
Network savings	—	60% reduction

5.3 Using a Combiner in Hadoop Streaming

For Word Count, the **reducer can also serve as the combiner** because addition is associative and commutative.

```
>_ Run Word Count with Combiner

# Run WITH combiner
hdfs dfs -rm -r /user/student/lab3/output_combined 2>/dev/null

hadoop jar $STREAMING_JAR \
  -input /user/student/lab3/input/wordcount_input.txt \
  -output /user/student/lab3/output_combined \
  -mapper "python3 mapper.py" \
  -combiner "python3 reducer.py" \
  -reducer "python3 reducer.py" \
  -file ~/mapper.py \
  -file ~/reducer.py

# View results (should be identical to without combiner)
hdfs dfs -cat /user/student/lab3/output_combined/part-00000
```

5.4 Compare Job Counters

> Check Job Statistics

```
# After each job, the console shows counters like:
#   Map input records
#   Map output records
#   Combine input records
#   Combine output records
#   Reduce input records

# You can also check via YARN Web UI:
# http://localhost:8088 -> click on your job -> Counters
```

Task 5.1 – Combiner Comparison

Run Word Count **without** and **with** a combiner, then fill in:

Metric	No Combiner	With Combiner
Map output records	56	56
Combine input records	N/A	56
Combine output records	N/A	39
Reduce input records	56	39

💡 Reflection 5.1

When can the reducer be reused as a combiner? When can it **not**? Give an example of each case.

The reducer can be reused as a combiner when the reduce function is associative and commutative, such as summation in WordCount.

It cannot be reused when the reduce function requires global information or performs non-associative operations, such as computing averages or medians.

Part 6: Deep Work – Advanced MapReduce Experiments (30 min)

🔧 About Deep Work: This section challenges you to go beyond the basics. You will modify programs, design experiments, and analyze MapReduce behavior. Deep work requires **critical thinking** — your reasoning must be supported by evidence.

6.1 Experiment 6.1: Inverted Index

An **inverted index** maps each word to the documents (or lines) it appears in. This is the foundation of search engines like Google.

```
>_ Create Multi-Document Input

# Create 3 separate "documents"
echo "hadoop mapreduce distributed computing cluster" > ~/doc1.txt
echo "spark hadoop big data analytics machine learning" > ~/doc2.txt
echo "mapreduce parallel processing big data hadoop" > ~/doc3.txt

# Upload to HDFS
hdfs dfs -mkdir -p /user/student/lab3/index_input
hdfs dfs -put ~/doc1.txt /user/student/lab3/index_input/
hdfs dfs -put ~/doc2.txt /user/student/lab3/index_input/
hdfs dfs -put ~/doc3.txt /user/student/lab3/index_input/
```

Task 6.1 – Build an Inverted Index

Write a mapper and reducer to produce an inverted index.

Expected output format:

```
analytics    doc2.txt
big          doc2.txt,doc3.txt
hadoop       doc1.txt,doc2.txt,doc3.txt
mapreduce    doc1.txt,doc3.txt
...
```

Hints:

- In Hadoop Streaming, the mapper can access the input filename via the environment variable: `os.environ['mapreduce_map_input_file']`
- Use `os.path.basename()` to extract just the filename
- Mapper output: `word\tfilename`
- Reducer: Collect unique filenames per word, join with commas

a) Write your inverted index mapper:

```
#!/usr/bin/env python3
import sys
import os

filename = os.path.basename(os.environ['mapreduce_map_input_file'])

for line in sys.stdin:
    words = line.strip().split()
    for word in words:
        print(f"{word}\t{filename}")
```

b) Write your inverted index reducer:

c) Show the output from Hadoop:

```

analytics doc2.txt
big doc2.txt, doc3.txt
cluster doc1.txt
computing doc1.txt
data doc2.txt, doc3.txt
distributed doc1.txt
hadoop doc1.txt, doc2.txt, doc3.txt
learning doc2.txt
machine doc2.txt
mapreduce doc1.txt, doc3.txt
parallel doc3.txt
processing doc3.txt
spark doc2.txt

```

```

#!/usr/bin/env python3
import sys

current_word = None
files = set()

for line in sys.stdin:
    word, filename = line.strip().split("\t")

    if word == current_word:
        files.add(filename)
    else:
        if current_word:
            print("{}\t{}".format(current_word, ','.join(sorted(files))))
        current_word = word
        files = {filename}

if current_word:
    print("{}\t{}".format(current_word, ','.join(sorted(files))))

```

Reflection 6.1

How does Google use an inverted index to answer a search query like “hadoop big data” in milliseconds? What role does MapReduce play in *building* the index vs. *querying* it?

Google uses an inverted index to map each word to documents containing it. When a user searches “hadoop big data”, Google quickly finds documents that contain those words by looking them up in the index. MapReduce is used to build the inverted index over massive datasets. Querying the index is fast and does not require MapReduce — it uses optimized search systems

6.2 Experiment 6.2: Finding Maximum & Minimum

```

>_ Create Temperature Dataset

# Create Saudi temperature data (city, month, temperature_C)
cat > ~/temperatures.csv << 'EOF'
Riyadh,Jan,15
Riyadh,Jul,45
Riyadh,Apr,30
Jeddah,Jan,25
Jeddah,Jul,40
Jeddah,Apr,32
Dammam,Jan,18
Dammam,Jul,43
Dammam,Apr,28
Abha,Jan,10
Abha,Jul,25
Abha,Apr,18
Tabuk,Jan,8
Tabuk,Jul,38
Tabuk,Apr,22
Riyadh,Feb,18
Jeddah,Feb,26
Dammam,Feb,20
Abha,Feb,12
Tabuk,Feb,10
EOF

hdfs dfs -mkdir -p /user/student/lab3/temp_input
hdfs dfs -put ~/temperatures.csv /user/student/lab3/temp_input/

```

Task 6.2 – Max/Min Temperature per City

Write a MapReduce program that outputs both the **maximum** and **minimum** temperature for each city.

Expected output format:

```
Abha      min=10,max=25
Dammam   min=18,max=43
Jeddah    min=25,max=40
Riyadh    min=15,max=45
Tabuk     min=8,max=38
```

a) Write your mapper and reducer code (attach or write below).

<pre> Reducer #!/usr/bin/env python3 import sys current_city = None total = 0 for line in sys.stdin: line = line.strip() if not line: continue city, temp = line.split("\t") temp = int(temp) if current_city == city: temp = append(temp) else: print(current_city, min(min(temp), max(max(temp)))) current_city = city temp = [temp] if current_city: print(current_city, min(min(temp), max(max(temp)))) </pre>	<pre> Mapper #!/usr/bin/env python3 import sys for line in sys.stdin: line = line.strip() if not line: continue parts = line.split("\t") if len(parts) != 2: continue city = parts[0].strip() temp = parts[1].strip() print(city + "\t" + temp) </pre>
---	---

b) Run on Hadoop and paste the output:

```

Abha      min=10,max=25
Dammam    min=18,max=43
Jeddah     min=25,max=40
Riyadh     min=15,max=45
Tabuk      min=8,max=38

```

Reflection 6.2

Can the reducer for max/min also be used as a combiner? Why or why not? Explain with an example.

Yes, the max/min reducer can be used as a combiner because max and min are associative and commutative operations. For example, $\max(\max(10,25), 18) = \max(10,25,18)$. Therefore, partial max/min values can safely be combined.

6.3 Experiment 6.3: Chaining MapReduce Jobs

Sometimes one MapReduce job is not enough. We can **chain** jobs: the output of Job 1 becomes the input of Job 2.

Task 6.3 – Two-Stage Pipeline

Using the Word Count output from Part 3:

Stage 1: Word Count (already done) → produces `word\tcount`

Stage 2: Write a new MapReduce program that reads the word count output and produces a **frequency distribution**: how many words appear exactly 1 time, 2 times, 3 times, etc.

Expected output format:

```

1    15    (15 words appeared exactly once)
2     8    (8 words appeared exactly twice)
3     3    (3 words appeared exactly 3 times)

```

Hints:

- Stage 2 mapper: swap – output `count\t1`
- Stage 2 reducer: sum the 1s for each count value
- Input for Stage 2 = output directory from Stage 1

a) Write your Stage 2 mapper and reducer.

```

Reducer
#!/usr/bin/env python3
import sys

current_count = None
total = 0

for line in sys.stdin:
    count, value = line.strip().split("\t")
    value = int(value)

    if count == current_count:
        total += value
    else:
        if current_count:
            print(current_count, total)
            current_count = count
            total = value

if current_count:
    print(current_count, total)

Mapper
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    word, count = line.strip().split("\t")
    print(count + "\t1")

```

- b) Show the commands to chain the two jobs.
c) Show the final frequency distribution output.

```
hadoop jar $STREAMING_JAR \
-input /user/student/lab3/output_no_combiner \
-output /user/student/lab3/frequency_output \
-mapper "python3 stage2_mapper.py" \
-reducer "python3 stage2_reducer.py" \
-file stage2_mapper.py \
-file stage2_reducer.py
```

The terminal window shows the output of the command: `hdfs dfs -cat /user/student/lab3/frequency_output`. The output is a frequency distribution of words from a text file, with the most frequent word being 'hadoop' (30 occurrences), followed by 'spark' (4), 'mapreduce' (3), 'hdfs' (1), and 'yarn' (1).

The Hadoop cluster status dashboard shows the following metrics:

Cluster Metrics	Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Memory
Cluster Nodes Metrics	6	0	0	0	0	0 B, 0 Cores

The dashboard also shows a table of running applications:

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	Start Time	Launch Time
application_1772108120573_0000	hams33443344	streaming42903344020851278.jar	MAPREDUCE		default	0	Sat Feb 28 11:18:07 +0300 2020	Sat Feb 28 11:18:11 +0300 2020
application_1772108120573_0005	hams33443344	streaming10240055330526851087.jar	MAPREDUCE		default	0	Sat Feb 28 11:11:12 +0300 2020	Sat Feb 28 11:11:19 +0300 2020

Reflection 6.3

What are the **limitations** of chaining MapReduce jobs? How does Apache Spark address these limitations? (Connect to what you learned in the lecture.)

Chaining MapReduce jobs causes multiple disk writes between stages, increasing latency. Apache Spark keeps data in memory between operations, reducing disk I/O and improving performance.

6.4 Experiment 6.4: Performance Analysis

Generate Large Input

```
# Generate a larger text file (~50 MB) for performance testing
python3 -c "
import random
words = ['hadoop', 'spark', 'mapreduce', 'hdfs', 'yarn', 'data',
         'big', 'analytics', 'cluster', 'distributed', 'node',
         'block', 'replica', 'namenode', 'datanode', 'computing',
         'machine', 'learning', 'pipeline', 'streaming', 'batch',
         'saudi', 'vision', '2030', 'effat', 'university']
with open('/tmp/large_input.txt', 'w') as f:
    for _ in range(500000):
        line = ' '.join(random.choices(words, k=random.randint(5,15)))
        f.write(line + '\n')
"

# Check size
ls -lh /tmp/large_input.txt

# Upload to HDFS
hdfs dfs -put /tmp/large_input.txt /user/student/lab3/input/
```

Task 6.4 – Performance Measurement

Run Word Count on the large file and record timing:

```
# Time the job execution
time hadoop jar $STREAMING_JAR \
-input /user/student/lab3/input/large_input.txt \
-output /user/student/lab3/perf_no_combiner \
-mapper "python3 mapper.py" \
```

```
-reducer "python3 reducer.py" \  
-file ~/mapper.py \  
-file ~/reducer.py
```

Then run again WITH a combiner and compare:

Metric	No Combiner	With Combiner
Execution time (real)	0m53s	1m22s
Map output records	10,000,007	10,000,007
Reduce input records	10,000,007	26
Bytes transferred (shuffle)	115,377,726	436

💡 Reflection 6.4

Did the combiner improve performance? Would the improvement be more significant on a real multi-node cluster? Why?

The combiner reduces shuffle data and improves performance. In a real multi-node cluster, the improvement would be more significant because network communication cost is higher across machines.

Part 7: Deliverables & Submission

(10 min)

 Submission Checklist

Item	Description	Marks
Task 1.1–1.2	YARN setup + screenshot + Streaming JAR path	1
Task 2.1	Pencil trace of MapReduce execution	2
Task 3.1	Local testing results and explanation	2
Task 3.2	Hadoop execution + YARN screenshot + output	3
Task 4.1	Saudi population per region results	2
Task 4.2	Custom hospital density MapReduce code + output	3
Task 5.1	Combiner comparison table	2
Reflection 5.1	When combiner = reducer (and when not)	1
Task 6.1	Inverted index code + output + reflection	3
Task 6.2	Max/min temperature code + output + reflection	2
Task 6.3	Chained jobs pipeline + output + reflection	3
Task 6.4	Performance comparison table + reflection	1
Total		25

Submission Instructions:

- Submit this completed lab document as a PDF with all screenshots, code, answers, and reflections.
- File name format: CS4074_Lab3_YourName_YourID.pdf
- Deadline: **One week from today's lab session.**
- Late submissions: **–5 marks per day.**

 Before You Leave

```
# Stop YARN and HDFS when done
stop-yarn.sh
stop-dfs.sh

# Verify all daemons are stopped
jps
```

 MapReduce Command Reference

Command	Description
<code>start-yarn.sh</code>	Start YARN ResourceManager & NodeManager
<code>stop-yarn.sh</code>	Stop YARN services
<code>hadoop jar \$STREAMING_JAR ...</code>	Run a Hadoop Streaming job
<code>-mapper "python3 mapper.py"</code>	Specify the mapper script
<code>-reducer "python3 reducer.py"</code>	Specify the reducer script
<code>-combiner "python3 reducer.py"</code>	Specify a combiner (optional)
<code>-file /script.py</code>	Ship local file to cluster nodes
<code>-input <hdfs_path></code>	Input directory in HDFS
<code>-output <hdfs_path></code>	Output directory in HDFS (must not exist)
<code>yarn application -list</code>	List running YARN applications
<code>yarn application -kill <id></code>	Kill a running application
<code>yarn logs -applicationId <id></code>	View application logs

End of Lab 3

Next Lab: Apache Spark RDDs
